École Nationale Supérieure de l'Intelligence Artificielle
et Sciences des Données — ENSIASD

Taroudant, Maroc

# SOUSS AMBULANCE

Système de Gestion des Urgences Médicales

## Rapport Technique de Projet

Module : Systèmes d'Information et Bases de Données (SIBD)

**Encadrante :** Pr. El Ateif Sara

**Réalisé par :** Siouda Nour
Souakh Rabab
Oukhouya Anas
Regragui Mohammed

**Filière :** 1ère année cycle Ingénieurs
Sciences des Données – Big Data & IA

**Année :** 2025–2026

# Table des matières


**1 Introduction Générale** **3**

1.1 Contexte et Problématique . . . . . 3

1.2 Objectifs du Projet . . . . . 3

1.3 Périmètre Fonctionnel . . . . . 4

**2 Analyse et Conception** **5**

2.1 Analyse des Besoins . . . . . 5

2.1.1 Acteurs du Système . . . . . 5

2.1.2 Flux Principal d'une Urgence . . . . . 5

2.2 Modèle Conceptuel des Données (MCD) . . . . . 5

2.2.1 Entités Principales . . . . . 6

2.2.2 Diagramme Entité-Association . . . . . 7

2.3 Modèle Logique des Données (MLD) . . . . . 8

**3 Implémentation de la Base de Données** **11**

3.1 Schéma Relationnel . . . . . 11

3.1.1 Table `incident` — Nœud de départ du cycle . . . . . 11

3.1.2 Table `mission` — Table Centrale . . . . . 11

3.1.3 Table `ambulance_position` — Traçabilité GPS . . . . . 11

3.1.4 Tables de Gestion Hospitalière . . . . . 11

3.2 Données Initiales . . . . . 11

**4 Procédures Stockées et Algorithme de Dispatch** **13**

4.1 Architecture des Procédures . . . . . 13

4.2 Algorithme de Sélection de l'Hôpital Optimal . . . . . 13

4.2.1 Formule de Score . . . . . 13

4.2.2 Implémentation SQL . . . . . 14

4.3 Procédure de Création Automatique de Mission . . . . . 14

**5 Triggers et Règles d'Intégrité** **15**

5.1 Vue d'Ensemble des Triggers . . . . . 15

5.2 Triggers Critiques . . . . . 15

5.2.1 Gestion Automatique des Lits Hospitaliers . . . . . 16

5.2.2 Mise à Jour Automatique du Statut de Mission . . . . . 16

5.2.3 Validation des Coordonnées GPS . . . . . 16

**6 Architecture Applicative** **17**

6.1 Architecture Générale . . . . . 17

6.2 API REST — Backend Flask . . . . . 17

6.2.1 Endpoints Implémentés . . . . . 18

6.2.2 Exemple d'Endpoint — Dispatch Automatique . . . . . 18

6.3 Interfaces Utilisateur . . . . . 18

6.3.1 Interface Dispatcher (`dispatcher.html`) . . . . . 19

6.3.2 Interface Chauffeur (`driver.html`) . . . . . 19

**7 Tests et Validation** **20**

7.1 Jeux de Tests . . . . . 20

7.1.1 Test de l'Algorithme de Dispatch . . . . . 20

7.1.2 Test des Triggers d'Intégrité . . . . . 20

7.2 Cas d'Utilisation — Scénario Complet . . . . . 21

**8 Bilan et Perspectives** **22**

8.1 Bilan Technique . . . . . 22

8.1.1 Réalisations . . . . . 22

8.1.2 Points Forts . . . . . 22

8.2 Améliorations Futures . . . . . 22

**Conclusion** **24**

# Chapitre 1

# Introduction Générale

## 1.1 Contexte et Problématique

La gestion des urgences médicales constitue un enjeu critique pour tout système de santé. Dans la région Souss-Massa, comme dans l'ensemble du territoire marocain, la coordination entre les appels d'urgence, le déploiement des ambulances et l'orientation vers les établissements hospitaliers adaptés repose encore largement sur des processus manuels, sources d'erreurs et de délais.

**Problématique**

Comment concevoir et implémenter un système d'information complet permettant d'automatiser le cycle de vie d'une urgence médicale — de la réception de l'appel jusqu'à l'admission du patient à l'hôpital — en optimisant l'affectation des ressources en temps réel ?

## 1.2 Objectifs du Projet

Le projet **Souss Ambulance** vise à construire une plateforme de coordination des urgences médicales articulée autour de trois axes :

1. **Modélisation relationnelle complète** : concevoir un schéma de base de données normalisé couvrant l'ensemble des entités métier (incidents, ambulances, hôpitaux, missions, patients).
2. **Logique métier en base** : implémenter des procédures stockées, des fonctions et des triggers PostgreSQL pour automatiser les règles de gestion.
3. **Interfaces utilisateur** : fournir une interface web pour les opérateurs de dispatch et une interface mobile pour les chauffeurs d'ambulance.

## 1.3 Périmètre Fonctionnel

Table 1.1 – Fonctionnalités principales du système

| **Module** | **Description** |
|---|---|
| Gestion des incidents | Création, classification par type et sévérité, suivi du statut |
| Dispatch automatique | Sélection de l'ambulance disponible et orientation vers le meilleur hôpital |
| Suivi GPS | Enregistrement continu de la position des ambulances |
| Gestion hospitalière | Suivi en temps réel de la capacité en lits par service |
| Gestion des missions | Cycle complet : dispatch → scène → hôpital → clôture |
| Tableau de bord | Statistiques en direct pour les opérateurs |

# Chapitre 2

# Analyse et Conception

## 2.1 Analyse des Besoins

### 2.1.1 Acteurs du Système

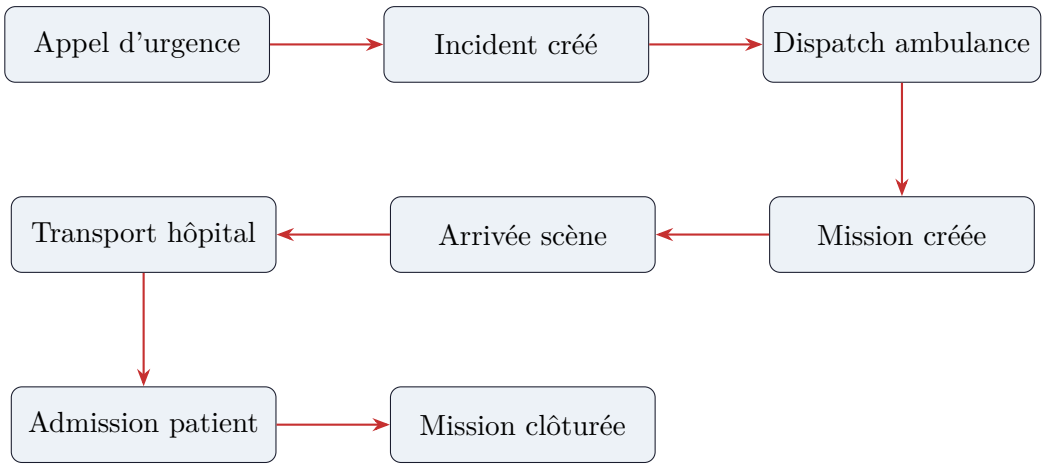
Trois types d'utilisateurs interagissent avec la plateforme :

— **Opérateur / Dispatcher** : reçoit les appels d'urgence, crée les incidents, déclenche les missions via l'interface `dispatcher.html`.

— **Chauffeur d'ambulance** : consulte sa mission active, met à jour les étapes de progression et partage sa position GPS via `driver.html`.

— **Administrateur système** : gère les référentiels (hôpitaux, types d'incidents, utilisateurs) via l'API REST.

### 2.1.2 Flux Principal d'une Urgence




## 2.2 Modèle Conceptuel des Données (MCD)

Le modèle conceptuel identifie **13 entités** organisées autour de la table centrale `mission`.

### 2.2.1 Entités Principales

Table 2.1 – Description des entités du système

| Entité | Clé primaire | Rôle |
|---|---|---|
| `app_user` | user_id | Utilisateurs du système (opérateurs, admins) |
| `incident_type` | incident_type_id | Référentiel des catégories d'urgence |
| `incident` | incident_id | Événement d'urgence déclaré |
| `emergency_call` | call_id | Appel entrant associé à un incident |
| `ambulance` | ambulance_id | Véhicule de secours avec position de base |
| `ambulance_position` | position_id | Historique GPS de chaque ambulance |
| `hospital` | hospital_id | Établissement hospitalier |
| `hospital_service` | service_id | Département hospitalier avec suivi de lits |
| `mission` | mission_id | Table centrale : lien incident-ambulance-hôpital |
| `routes` | route_id | Données de trajet calculées pour une mission |
| `patients` | patient_id | Informations patient anonymisées |
| `admissions` | admission_id | Admission d'un patient dans un service |
| `audit_logs` | audit_id | Journal de traçabilité des actions |

### 2.2.2 Diagramme Entité-Association

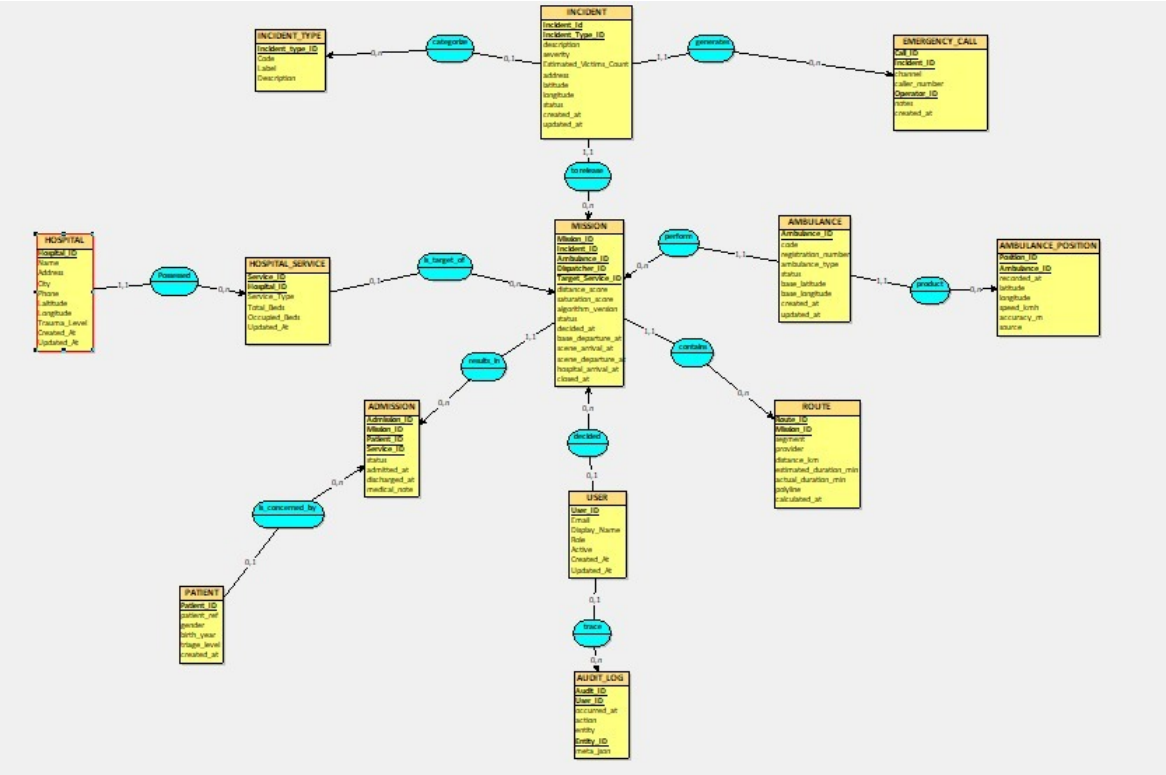



FIGURE 2.1 – MCD : Modèle Conceptuel de Données Souss Ambulance

## 2.3 Modèle Logique des Données (MLD)

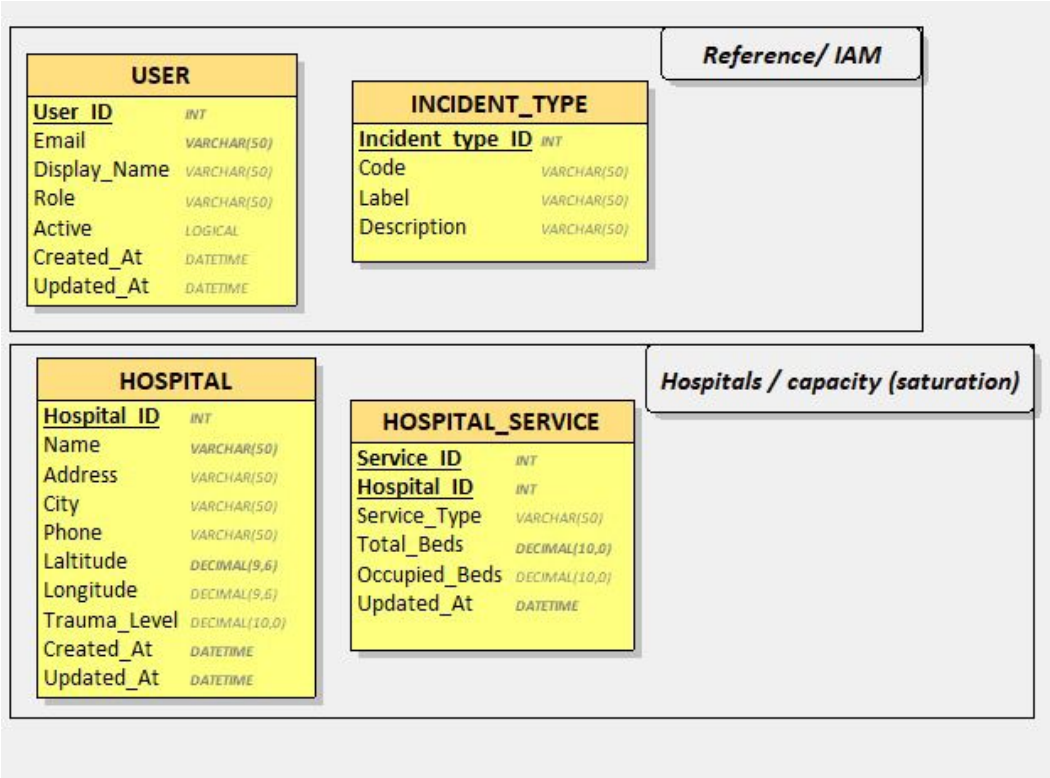



FIGURE 2.2 – MLD - Référentiels (IAM) et Gestion de la Capacité Hospitalière

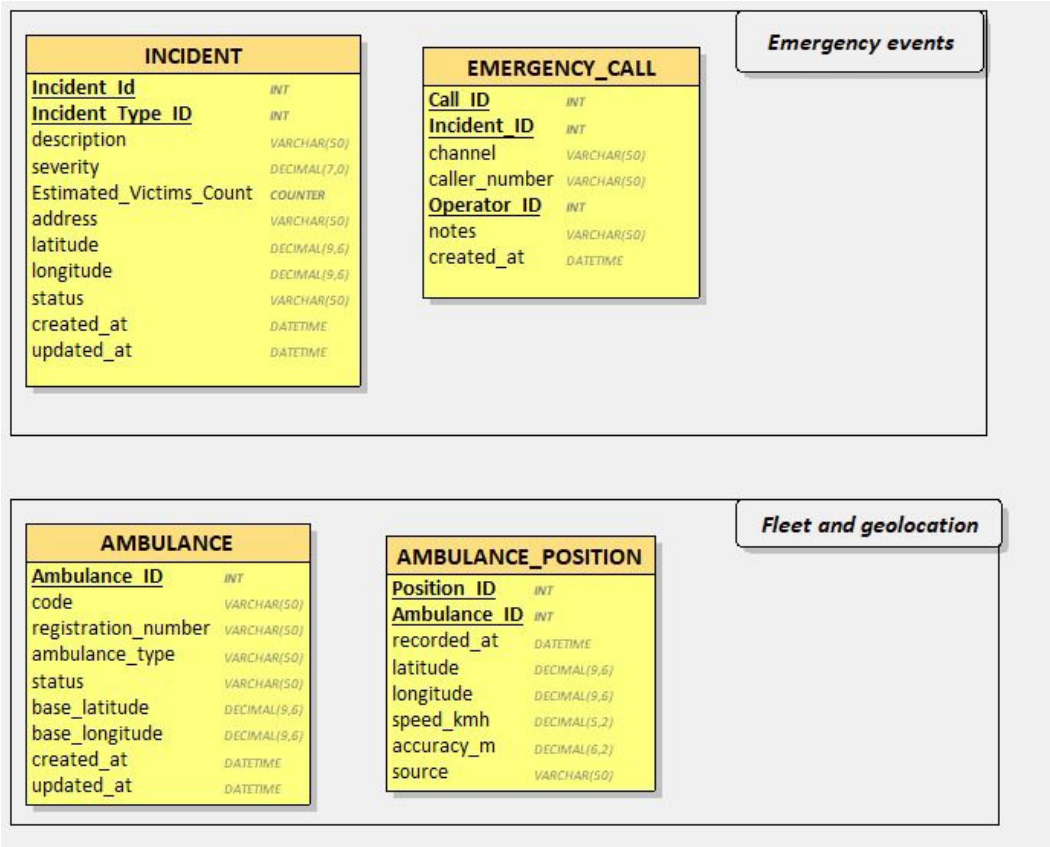



FIGURE 2.3 – MLD - Événements d'Urgence et Flotte d'Ambulances

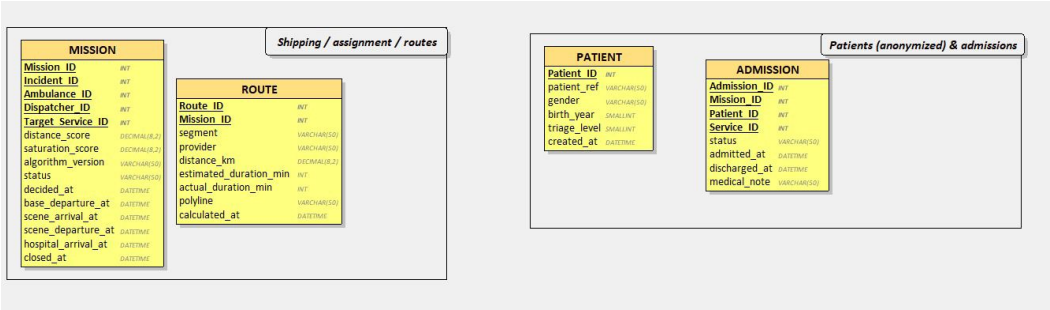



FIGURE 2.4 – MLD - Missions, Routage et Admissions des Patients

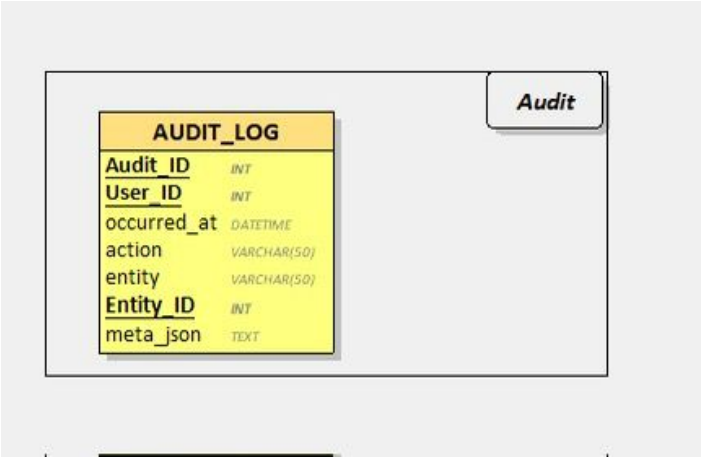



FIGURE 2.5 – MLD - Journal d'Audit Système

# Chapitre 3

# Implémentation de la Base de Données

## 3.1 Schéma Relationnel

La base de données **souss_ambulance** est implémentée sous **PostgreSQL**. Le schéma respecte la **3ème forme normale (3FN)**.

### 3.1.1 Table `incident` — Nœud de départ du cycle

Cette table stocke les informations essentielles de l'urgence : type, localisation GPS, adresse, sévérité estimée et le statut actuel de l'événement.

### 3.1.2 Table `mission` — Table Centrale

Cette table matérialise l'association entre un incident, une ambulance dépêchée et un hôpital de destination. Elle enregistre de plus tous les horodatages du cycle d'intervention et les scores algorithmiques ayant mené aux décisions de dispatching.

### 3.1.3 Table `ambulance_position` — Traçabilité GPS

Conçue pour stocker l'historique de géolocalisation des flottes, elle permet d'enregistrer les coordonnées, la vitesse, la précision du signal et la méthode de capture (automatique ou manuelle) pour chaque ambulance.

### 3.1.4 Tables de Gestion Hospitalière

Composées des entités `hospital` et `hospital_service`, elles gèrent la cartographie des hôpitaux et le taux d'occupation des lits par service, intégrant des contraintes de base de données pour empêcher la surréservation.

## 3.2 Données Initiales

Afin de valider le fonctionnement, un ensemble de données initiales est intégré. Celui-ci comprend des points d'intérêts critiques (comme l'Hôpital Hassan II à Agadir),

la répartition des lits d'urgence et une taxonomie complète de 9 types d'incidents (accident de la route, malaise cardiaque, noyade, etc.).

# Chapitre 4

# Procédures Stockées et Algorithme de Dispatch

## 4.1 Architecture des Procédures

La logique métier est encapsulée dans **trois fonctions PL/pgSQL** qui couvrent les besoins critiques du système.

TABLE 4.1 – Fonctions stockées implémentées

| Fonction | Rôle | Retour |
|---|---|---|
| `find_best_hospital` | Sélection de l'hôpital optimal selon un score multicritère | TABLE |
| `estimate_route` | Estimation de la distance et du temps de trajet | TABLE |
| `create_mission_auto` | Création automatique d'une mission complète | VOID |

## 4.2 Algorithme de Sélection de l'Hôpital Optimal

### 4.2.1 Formule de Score

La fonction calcule un **score composite** combinant deux critères :

$$\text{Score} = \underbrace{\left(1 - \frac{\text{lits_occupés}}{\text{lits_totaux}}\right) \times 0{,}6}_{\text{disponibilité (60 \%)}} + \underbrace{\frac{1}{1 + |\Delta\text{lat}| + |\Delta\text{lon}|} \times 0{,}4}_{\text{proximité (40 \%)}} \tag{4.1}$$

— **Critère disponibilité (60 %)** : favorise les services hospitaliers ayant le plus grand ratio de lits libres.

— **Critère proximité (40 %)** : favorise les hôpitaux les plus proches du lieu de l'incident (distance de Manhattan normalisée).

### 4.2.2 Implémentation SQL

Le processus filtre d'abord les services saturés via une clause restrictive, avant de procéder au tri par pertinence du score défini ci-dessus. L'hôpital affichant le meilleur compromis disponibilité/distance est retourné automatiquement.

## 4.3 Procédure de Création Automatique de Mission

Lors de la génération d'une mission, la procédure recherche d'abord l'hôpital optimal en invoquant l'algorithme de dispatching. Elle insère ensuite l'enregistrement au sein de la table de mission avec les identifiants requis et un statut initial "DISPATCHED", enregistrant simultanément la date et l'heure de la décision.

# Chapitre 5

# Triggers et Règles d'Intégrité

## 5.1 Vue d'Ensemble des Triggers

Le système implémente **7 triggers** couvrant la validation, l'automatisation et l'audit.

Table 5.1 – Inventaire des triggers PostgreSQL

| **Trigger** | **Table** | **Événement** | **Action** |
|---|---|---|---|
| `trg_admission_beds` | admissions | INSERT / UPDATE | Incrémente / décrémente `occupied_beds` |
| `trg_capacity` | admissions | BEFORE INSERT | Bloque si service plein |
| `trg_mission_status` | mission | BEFORE UPDATE | Met à jour `status` automatiquement |
| `trg_ambulance_check` | mission | BEFORE INSERT | Bloque si ambulance non disponible |
| `trg_ambulance_busy` | mission | AFTER INSERT | Passe l'ambulance en statut `BUSY` |
| `trg_gps_validation` | ambulance_position | BEFORE INSERT | Valide les coordonnées GPS |
| `trg_mission_audit` | mission | AFTER INSERT/UPDATE | Enregistre dans `audit_logs` |

## 5.2 Triggers Critiques

### 5.2.1 Gestion Automatique des Lits Hospitaliers

Un trigger veille à la mise à jour dynamique des capacités du service de destination. Dès qu'un patient est inséré avec le statut 'ADMITTED', le compteur des lits occupés s'incrémente. Lorsqu'il passe en statut 'DISCHARGED', le lit est de nouveau libéré.

### 5.2.2 Mise à Jour Automatique du Statut de Mission

Afin de minimiser les interventions manuelles, l'état de la mission ('ON_SITE', 'AT_HOSPITAL', 'COMPLETED') évolue dynamiquement en fonction de la complétion des champs d'horodatages correspondants aux différentes étapes de l'opération.

### 5.2.3 Validation des Coordonnées GPS

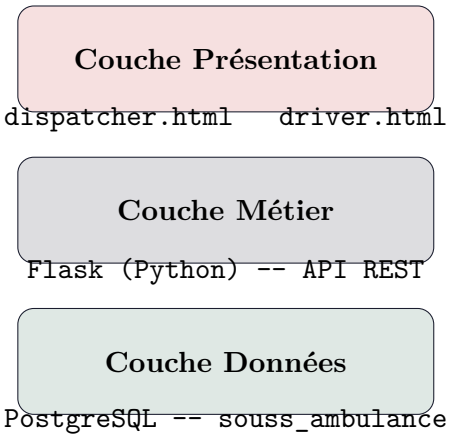
Une protection de base s'assure que toutes les coordonnées géographiques intégrées dans le système respectent les limites physiques (latitude de -90 à 90, et longitude de -180 à 180), renvoyant une exception lors d'une saisie aberrante.

# Chapitre 6

# Architecture Applicative

## 6.1 Architecture Générale

Le système adopte une architecture **3-tiers** classique :




## 6.2 API REST — Backend Flask

### 6.2.1 Endpoints Implémentés

Table 6.1 – Endpoints de l'API REST

| Méthode | Route | Description |
|---|---|---|
| GET | `/api/stats` | Statistiques du tableau de bord |
| GET | `/api/incidents` | Liste de tous les incidents |
| POST | `/api/incidents` | Créer un nouvel incident |
| GET | `/api/ambulances` | Liste de toutes les ambulances |
| GET | `/api/ambulances/available` | Ambulances disponibles uniquement |
| POST | `/api/ambulances` | Ajouter une ambulance |
| POST | `/api/ambulances/:id/position` | Mettre à jour position GPS |
| GET | `/api/missions` | Liste de toutes les missions |
| POST | `/api/missions/auto` | Créer mission (appel procédure DB) |
| PATCH | `/api/missions/:id/status` | Avancer une étape de mission |
| GET | `/api/hospitals` | Hôpitaux avec services et lits |
| POST | `/api/hospitals` | Ajouter un hôpital |
| POST | `/api/admissions` | Admettre un patient |
| PATCH | `/api/admissions/:id/discharge` | Sortie de patient |
| GET | `/api/incident-types` | Référentiel des types d'incidents |

### 6.2.2 Exemple d'Endpoint — Dispatch Automatique

L'interface de programmation Flask gère l'orchestration lors de la réception d'une requête POST de création de mission. Le backend relaie les identifiants d'incident, d'ambulance, ainsi que les données de localisation vers la base de données PostgreSQL, puis acquitte l'opération tout en mettant à jour le statut global de l'incident.

## 6.3 Interfaces Utilisateur

### 6.3.1 Interface Dispatcher (`dispatcher.html`)

L'interface dispatcher est une **Single Page Application** développée en HTML/CSS/JavaScript vanilla, accessible depuis un navigateur de bureau.

Elle comprend cinq onglets :

- **Dashboard** : indicateurs clés temps réel (missions actives, ambulances disponibles, incidents ouverts, lits disponibles/occupés), actualisés régulièrement.
- **Incidents** : liste complète avec formulaire de création et bouton de dispatch direct.
- **Missions** : suivi de toutes les missions en cours avec avancement par étapes.
- **Ambulances** : gestion du parc avec statuts en temps réel.
- **Hôpitaux** : capacité par service, taux d'occupation.

### 6.3.2 Interface Chauffeur (`driver.html`)

Interface mobile optimisée (max-width : 480 px) destinée aux chauffeurs d'ambulance :

- Sélection de l'ambulance assignée au chauffeur.
- Affichage de la **mission active** avec les étapes à valider séquentiellement :

  `Départ base` → `Arrivée scène` → `Départ scène` → `Arrivée hôpital` → `Clôture`

- **Partage GPS automatique** via l'API `Geolocation` du navigateur (`watchPosition`) avec envoi automatique à chaque déplacement.
- Historique des dernières missions de l'ambulance.

# Chapitre 7

# Tests et Validation

## 7.1 Jeux de Tests

### 7.1.1 Test de l'Algorithme de Dispatch

TABLE 7.1 – Scénario de test — Sélection de l'hôpital optimal

| **Paramètre** | **Valeur** | **Résultat attendu** |
|---|---|---|
| Incident à | 30,4278°N, −9,5981°E (Agadir Centre) | — |
| Hôpital A | Urgences : 5/20 lits occupés, distance 0 | **Sélectionné** |
| Hôpital B | Urgences : 20/20 lits occupés | Exclu (plein) |

$$\text{Score A} = \left(1 - \frac{5}{20}\right) \times 0{,}6 + \frac{1}{1+0} \times 0{,}4 = 0{,}75 \times 0{,}6 + 1 \times 0{,}4 = \mathbf{0{,}85}$$

### 7.1.2 Test des Triggers d'Intégrité

TABLE 7.2 – Cas de test — Triggers

| **Test** | **Action** | **Comportement** |
|---|---|---|
| Admission service plein | INSERT dans `admissions` si `occupied = total` | Exception levée ✓ |
| Ambulance occupée | Créer une mission avec ambulance `BUSY` | Exception levée ✓ |
| GPS invalide | Insérer position avec latitude = 200° | Exception levée ✓ |
| Statut mission auto | UPDATE `arrive_scene_at = NOW()` | Statut → `ON_SITE` ✓ |
| Lits auto | Admission créée | `occupied_beds + 1` ✓ |

## 7.2 Cas d'Utilisation — Scénario Complet

**Scénario :** Un accident de la route survient Avenue Hassan II, Agadir. L'opérateur crée l'incident, sélectionne l'ambulance AMB-01, lance la mission automatique. L'hôpital Hassan II (service URGENCES, 15 lits libres) est sélectionné. Le chauffeur valide chaque étape depuis son mobile. Le patient est admis et le lit est comptabilisé automatiquement.

**Étape 1.** `POST /api/incidents` ⇒ incident_id = 1, status = OPEN

**Étape 2.** `POST /api/missions/auto` ⇒ `create_mission_auto(1, 1, 30.4278, -9.5981)`

- `find_best_hospital()` retourne service_id = 1 (URGENCES, score = 0,85)
- Mission créée : status = DISPATCHED
- Incident mis à jour : status = ASSIGNED
- Ambulance mise à jour : status = BUSY (trigger)

**Étape 3.** `PATCH /api/missions/1/status` {arrive_scene_at} ⇒ trigger → `ON_SITE`

**Étape 4.** `PATCH /api/missions/1/status` {arrive_hospital_at} ⇒ trigger → `AT_HOSPITAL`

**Étape 5.** `POST /api/admissions` ⇒ trigger → occupied_beds = 6

**Étape 6.** `PATCH /api/missions/1/status` {closed_at} ⇒ trigger → `COMPLETED`

# Chapitre 8

# Bilan et Perspectives

## 8.1 Bilan Technique

### 8.1.1 Réalisations

Table 8.1 – Synthèse des livrables

| Livrable | Statut | Détail |
|---|---|---|
| Schéma de base de données | ✓ | 13 tables, contraintes d'intégrité, 3FN |
| Données de test | ✓ | Hôpitaux, services, 9 types d'incidents |
| Procédures stockées | ✓ | 3 fonctions PL/pgSQL |
| Triggers | ✓ | 7 triggers (validation, automatisation, audit) |
| API REST | ✓ | 16 endpoints Flask/Python |
| Interface Dispatcher | ✓ | SPA web, 5 modules, rafraîchissement auto |
| Interface Chauffeur | ✓ | Interface mobile, GPS, suivi mission |

### 8.1.2 Points Forts

— **Logique encapsulée en base** : l'algorithme de dispatch et les règles d'intégrité résident dans la base de données, garantissant leur application indépendamment du client.

— **Automatisation par triggers** : la gestion des lits, les statuts de mission et l'audit sont entièrement automatiques, sans code applicatif supplémentaire.

— **Architecture découplée** : le backend REST permet l'ajout futur d'applications mobiles natives.

## 8.2 Améliorations Futures

1. **Authentification JWT** : sécuriser l'API avec des tokens par rôle (dispatcher, driver, admin).

2. **Calcul de trajet réel** : intégrer l'API OpenRouteService (ORS) pour remplacer la distance de Manhattan par un itinéraire routier réel.
3. **Notifications temps réel** : utiliser WebSockets (Flask-SocketIO) pour les alertes push vers les chauffeurs.
4. **Cartographie interactive** : intégrer Leaflet.js avec les tuiles OpenStreetMap pour visualiser ambulances et incidents sur une carte.
5. **Statistiques avancées** : tableaux de bord analytiques (temps de réponse moyen, taux d'occupation hospitalier par période).
6. **Gestion des équipes médicales** : suivi du personnel SMUR par ambulance.

# Conclusion

Le projet **Souss Ambulance** démontre l'importance de placer la logique métier au cœur du système de base de données plutôt que dans la couche applicative. L'utilisation des procédures stockées PL/pgSQL pour l'algorithme de dispatch, et des triggers pour l'intégrité référentielle et l'automatisation, garantit la cohérence des données quelle que soit l'interface cliente.

Le schéma relationnel en **3ème forme normale**, combiné aux **contraintes CHECK**, aux **clés étrangères** et aux **7 triggers**, constitue un système robuste capable de gérer le cycle de vie complet d'une urgence médicale — de l'appel initial à la sortie du patient — avec une traçabilité totale via le journal d'audit.

Ce projet représente une base solide pour un déploiement réel dans le contexte des services d'urgence de la région Souss-Massa.

---

*Rapport réalisé dans le cadre du module Systèmes d'Information et Bases de Données (SIBD)*

ENSIASD — Taroudant, Maroc, 2026